

Adaptive-HISA Weighted Select 的两项 Warp 级优化

Warp-Aggregated Weighted Histogram + Ballot-based Threshold Bucket Compaction

面向 `weighted_select(1).cu` 的 Cursor 代码修改说明

文档用途：可直接作为 Cursor 的实现输入。目标是在不改变 `weighted selection` 语义、`threshold` 结果、`stable tie` 规则和 `CUDA Graph` 可捕获性的前提下，减少 `shared-memory atomicAdd` 的竞争，并把 `threshold bucket compaction` 从“每个匹配 leaf 一次 `atomicAdd`”改成“每个 warp 每轮最多一次 `atomicAdd`”。

1. 修改范围与不可破坏的语义

本次只修改生产 fast path: `weighted_prefix_fast_kernel`。fallback、最终 `logical-order scan`、`expand kernel` 和 `host` 接口先保持不变，便于做 `bit-exact A/B` 对照。

项目	要求
必须保持	leaf 按 score 降序、相同 score 按 leaf index 升序；budget 按 clipped leaf length 计权；crossing leaf 可部分选取。
必须保持	<code>ordered_float_key()</code> 的 FP32 全序语义；-0/+0 处理不变。
必须保持	Phase A 找到的 <code>threshold high-byte</code> 、Phase C 最终 <code>threshold_key</code> 、 <code>candidate_count</code> 、 <code>selected_prefix</code> 与基线一致。
必须保持	fast path capacity <= 4096 时 <code>threshold_candidates</code> 容量足够；不得因为聚合优化改变 <code>candidate_count</code> 的真实计数。
允许改变	<code>threshold_candidates</code> 中候选 leaf 的物理排列顺序，因为后续 Phase C 只做加权 histogram，不依赖 <code>candidate</code> 数组顺序。

当前代码的两个热点都位于 fast path：Phase A 对 256-bin shared histogram 做逐 leaf `atomicAdd`；Phase B 对 `threshold high-byte bucket` 的每个匹配 leaf 做一次 `candidate_count atomicAdd`。源代码中 Phase C 的后续三个 `radix byte refinement` 也重复使用逐 candidate shared `atomicAdd`，因此建议同一并改。

```
// 当前 Phase A
if (length > 0) {
    const int bin = static_cast<int>(ordered_float_key(scores[index]) >> 24);
```

```

    atomicAdd(&histogram[bin], static_cast<uint32_t>(length));
}

// 当前 Phase B
if (static_cast<int32_t>(key >> 24) == threshold_byte) {
    const int pos = atomicAdd(&candidate_count, 1);
    if (pos < kFastMaxLeaves) {
        threshold_candidates[pos] =
            (static_cast<uint64_t>(static_cast<uint32_t>(length)) << 32) |
            static_cast<uint32_t>(index);
    }
}
}

```

2. 优化一：Warp-Aggregated Weighted Histogram

目标：把“一个 leaf 一次 shared atomicAdd”改为“同一 warp 内、落入同一个 radix bin 的 leaf 先做权重求和，再由该组一个 leader 执行一次 atomicAdd”。对于 score 高度聚集的 weighted radix histogram，这能显著减少同一 shared 地址上的原子竞争。

NVIDIA 架构依据：__match_any_sync 可把一个 warp 中具有相同 bin 值的 lane 组成 peer mask；在 compute capability 8.x 及以上可用 __reduce_add_sync(peer_mask, weight) 直接做该 peer group 的整数加和。__match_any_sync 自 7.x 起支持，而 warp reduce intrinsic 自 8.x 起支持，因此建议提供 < 8.0 的 shuffle fallback。

2.1 新增两个 device helper

建议把 helper 放在 ordered_float_key()/stable_leaf_key() 附近。第一层 helper 对任意 peer mask 求 uint32 权重和；第二层 helper 完成 match-any + group leader atomic。

```

__device__ __forceinline__ uint32_t warp_group_sum_u32(
    unsigned peer_mask, uint32_t value) {
    #if defined(__CUDA_ARCH__) && (__CUDA_ARCH__ >= 800)
        return __reduce_add_sync(peer_mask, value);
    #else
        // 所有 peer lanes 必须执行同样的 shuffle 序列。
        uint32_t sum = 0;
        unsigned lanes = peer_mask;
        while (lanes != 0) {
            const int src = __ffs(lanes) - 1;
            sum += __shfl_sync(peer_mask, value, src);
            lanes &= lanes - 1;
        }
        return sum;
    #endif
}

__device__ __forceinline__ void warp_aggregated_hist_add(
    uint32_t *histogram, bool valid, int bin, uint32_t weight) {
    const unsigned active = __activemask();
    const unsigned valid_mask = __ballot_sync(active, valid);
    if (!valid) {
        return;
    }

    // valid_mask 中所有同 bin lanes 得到完全相同的 peer_mask。
    const unsigned peer_mask = __match_any_sync(valid_mask, bin);
    const uint32_t group_weight = warp_group_sum_u32(peer_mask, weight);
    const int lane = static_cast<int>(threadIdx.x) & 31;
    const int leader = __ffs(peer_mask) - 1;

    if (lane == leader) {

```

```
    atomicAdd(&histogram[bin], group_weight);
  }
}
```

2.2 替换 Phase A 的 weighted histogram

不要在 `length <= 0` 时 `continue` 后再调用 `warp intrinsic`；应先形成 `valid`，然后让同一轮仍处于 `active` 的 `warp lanes` 进入 `helper`。这样 `sync mask` 是明确且一致的。

```
// BEFORE
for (int index = tid; index < n; index += kThreads) {
    int32_t first;
    const int32_t length = clipped_leaf(starts[index], lengths[index], g, &first);
    if (length > 0) {
        const int bin = static_cast<int>(ordered_float_key(scores[index]) >> 24);
        atomicAdd(&histogram[bin], static_cast<uint32_t>(length));
    }
}

// AFTER
for (int index = tid; index < n; index += kThreads) {
    int32_t first;
    const int32_t length = clipped_leaf(starts[index], lengths[index], g, &first);
    const bool valid = length > 0;
    const uint32_t key = valid ? ordered_float_key(scores[index]) : 0u;
    const int bin = static_cast<int>(key >> 24);
    const uint32_t weight = valid ? static_cast<uint32_t>(length) : 0u;

    warp_aggregated_hist_add(histogram, valid, bin, weight);
}
```

2.3 同样替换 Phase C 的三个 radix refinement pass

Phase C 当前对 `threshold_candidates` 中每个仍匹配 `prefix` 的 `candidate` 做一次 `shared atomicAdd`。这里也适合使用完全相同的 `helper`，否则 Phase A 原子数下降后，Phase C 可能成为新的 `shared-atomic` 热点。

```
// BEFORE
if ((key & prefix_mask) == threshold_key) {
    const int bin = static_cast<int>((key >> shift) & 0xffu);
    atomicAdd(&histogram[bin], weight);
}

// AFTER
const bool valid = ((key & prefix_mask) == threshold_key);
const int bin = static_cast<int>((key >> shift) & 0xffu);
warp_aggregated_hist_add(histogram, valid, bin, weight);
```

2.4 为什么结果保持完全一致

原算法对 `histogram[bin]` 做的是整数权重加法。优化后只是把同一 `warp`、同一 `bin` 的若干次 `atomicAdd` 先在 `warp` 内求和，再执行一次 `atomicAdd`。加法对象和总权重都不变，因此每个 `bin` 的最终 `uint32` 计数应完全一致；后面的 `threshold_byte`、`remaining`、`threshold_key` 也应一致。

场景	原 atomic 数	优化后 atomic 数（上界/直觉）
warp 32 个 lane 全落不同 bin	32	32，基本无收益
32 个 lane 全落同一个 bin	32	1，最大约 32x 减少
32 个 lane 落入 4 个 bin	32	约 4

场景	原 atomic 数	优化后 atomic 数（上界/直觉）
大量 score 聚集在少数高字节 bin	大量竞争	通常明显下降

注意：不要把此优化和 per-warp private histogram 同时一次性改入第一版。先单独实现 warp aggregation，保持 shared histogram[256] 不变，这样容易证明正确性和定位性能收益。

3. 优化二：Threshold bucket compaction 用 ballot + warp prefix

目标：当前 Phase B 中，每个命中 threshold_byte 的 leaf 都执行 `atomicAdd(&candidate_count, 1)`。改为每个 warp 先 ballot 出本轮命中的 lanes，一个 warp 只做一次 `atomicAdd(candidate_count, warp_match_count)`，再由 lane-local prefix 计算每个候选在 threshold_candidates 中的位置。

3.1 新增 warp compaction helper（推荐内联到 Phase B）

```
__device__ __forceinline__ int warp_compact_reserve(
    bool match, int32_t *global_count, int *warp_count_out) {
    const unsigned active = __activemask();
    const unsigned match_mask = __ballot_sync(active, match);
    const int lane = static_cast<int>(threadIdx.x) & 31;
    const int warp_count = __popc(match_mask);
    *warp_count_out = warp_count;

    if (warp_count == 0) {
        return -1;
    }

    const int leader = __ffs(match_mask) - 1;
    int base = 0;
    if (lane == leader) {
        base = atomicAdd(global_count, warp_count);
    }
    // 所有 active lanes 都执行 broadcast, source lane 一定 active。
    base = __shfl_sync(active, base, leader);

    if (!match) {
        return -1;
    }

    const unsigned lanes_before =
        (lane == 0) ? 0u : ((1u << lane) - 1u);
    const int local_rank = __popc(match_mask & lanes_before);
    return base + local_rank;
}
```

3.2 重写 Phase B，禁止 continue 跳过 ballot

关键点：原代码 `length <= 0` 会 continue。改造后必须让本轮 active warp lanes 一起执行 ballot，所以应把条件折叠进 match 布尔值，不要在 ballot 前做数据依赖的 continue。

```
// AFTER: Phase B
for (int index = tid; index < n; index += kThreads) {
    int32_t first;
    const int32_t length = clipped_leaf(starts[index], lengths[index], g, &first);
    const bool valid = length > 0;
    const uint32_t key = valid ? ordered_float_key(scores[index]) : 0u;
    const bool match = valid &&
        (static_cast<int32_t>(key >> 24) == threshold_byte);
```

```

int warp_count = 0;
const int pos = warp_compact_reserve(match, &candidate_count, &warp_count);

if (match && pos < kFastMaxLeaves) {
    threshold_candidates[pos] =
        (static_cast<uint64_t>(static_cast<uint32_t>(length)) << 32) |
        static_cast<uint32_t>(index);
}
}
__syncthreads();

```

3.3 为什么 candidate 数组顺序改变也没关系

原实现中 candidate_count 的 atomicAdd 已经不能保证严格的 leaf-index 排列；新实现会按 “warp reservation + lane prefix” 形成另一种物理顺序。Phase C 只读取每个 packed record 的 (weight,index)，重新从 scores[index] 取 key 并做 histogram，因此 candidate 数组顺序不参与 selection 语义。最终 stable tie 仍由后面的 logical-order scan 用 leaf index 顺序完成。

3.4 原子操作数量的预期变化

若 threshold bucket 中有 M 个 leaf，旧实现需要近似 M 次 candidate_count atomicAdd。新实现每个 warp 的每个循环轮次最多一次 atomicAdd，因此近似为：**Atomic_new $\approx$ 活跃 warp 轮次，而不是 M**

例如 kThreads=1024、threshold bucket 较密且一次扫描中大多数 lanes 都 match 时，理论上可以把 1024 次 atomicAdd 降到约 32 次；若只有少量零散匹配，则收益相应变小。

4. 推荐 Cursor 的实际修改顺序

- Step 1** - 只新增 warp_group_sum_u32() 和 warp_aggregated_hist_add()，仅替换 Phase A。编译并跑 bit-exact 测试。
- Step 2** - 把 Phase C 三个 radix refinement 的 shared histogram atomicAdd 也改成 warp_aggregated_hist_add()。再次验证 threshold_key 与 selected_prefix。
- Step 3** - 新增 warp_compact_reserve()，重写 Phase B threshold bucket packing。不得使用 length<=0 的 continue 跳过 ballot。
- Step 4** - 保持 fallback kernel、final logical scan、expand kernel 不变。先确认为行为与性能，再考虑后续 1024->256/512 threads、pair scan 等其他优化。

5. 正确性验证要求（Cursor 必须完成）

这两个优化都属于 “执行方式改变、selection contract 不变” 的优化，因此测试目标应该是 bit-exact / element-exact，而不是只看最终 recall。

- 对同一输入同时运行旧 kernel 与新 kernel，逐元素比较 selected_prefix；随后比较 expand 后 candidates 与 count。
- 记录并比较 threshold_byte、threshold_key、candidate_count、remaining（可在 debug build 中临时输出到 workspace）。
- 覆盖 num_leaves = 0、1、31、32、33、1023、1024、1025、4096。

- 覆盖所有 length 相同、length 为 0、guard 截断 leaf、crossing leaf、budget 小于 guard、budget 大于全部 non-guard token。
- 覆盖大量 score 相同（测试 stable tie）、+0/-0、正负 score、极小/极大有限 FP32；若生产输入保证无 NaN/Inf，也至少确认 helper 不改变已有输入域。
- 用随机种子批量生成至少数千组 scores/starts/lengths/budget，与原 fast path 以及 reference select_candidates 双重对照。
- 在 CUDA Graph capture/replay 下重复运行，确认无动态分配、无 host sync、结果稳定。

6. 性能验证与判定标准

建议用 CUDA event 记录 weighted_prefix_fast_kernel 单 kernel 延迟，同时保留现有 profile_stats 的 threshold-bucket ratio。性能测试要按 bucket ratio 分组，因为两项优化的收益高度依赖 score 聚集程度。

指标	建议记录	判定
kernel latency	P50/P90/P99, graph on/off	新版本应稳定下降，不能只在单一随机输入下降
threshold bucket ratio	candidate_count / n	按 <5%、5-20%、20-50%、>50% 分桶观察收益
candidate_count	旧/新逐次一致	必须一致
selected_prefix/candidates	逐元素一致	必须完全一致
shared atomic 压力	Nsight Compute 的 shared atomic/serialization 相关 section	Phase A/C 应下降
candidate_count atomic	可用 debug counter 统计 warp leader reserve 次数	应从 O(M) 接近 O(warp rounds)

7. Cursor 可直接使用的修改指令

请修改 weighted_select(1).cu 的 weighted_prefix_fast_kernel，只做以下两项优化，不要改变 selection contract、host API、fallback kernel、final logical-order scan 和 expand kernel：

- 1) Warp-Aggregated Weighted Histogram
 - 新增 warp_group_sum_u32(peer_mask, value)。
 - sm80+ 使用 __reduce_add_sync；sm70-sm75 使用所有 peer lanes 共同执行的 __shfl_sync fallback。
 - 新增 warp_aggregated_hist_add(histogram, valid, bin, weight):
 - a. active = __activemask()
 - b. valid_mask = __ballot_sync(active, valid)
 - c. valid lanes 用 __match_any_sync(valid_mask, bin) 得到 peer_mask
 - d. 对 peer group 的 weight 求和
 - e. peer leader 对 histogram[bin] 只做一次 atomicAdd
 - 用该 helper 替换 Phase A 的逐-leaf atomicAdd。
 - 用同一 helper 替换 Phase C 三个 radix refinement pass 的逐-candidate atomicAdd。
- 2) Threshold bucket ballot compaction
 - 新增 warp_compact_reserve(match, &candidate_count, ...)。
 - 每个 warp 用 __ballot_sync 得到 match_mask，__popc 得到 warp_count。

- match_mask leader 对 candidate_count 只做一次 atomicAdd(warp_count)。
- 用 __shfl_sync 广播 base。
- 每个 match lane 用 popc(match_mask & lanes_before) 得到 local_rank, pos = base + local_rank。
- Phase B 中删除 length<=0 的 continue; 把它折叠到 match=false, 确保同一 active warp 的 lanes 一起执行 ballot。
- threshold_candidates[pos] 的 packed layout 保持原样: 高 32 位=length, 低 32 位=index。
- candidate_count 必须仍等于 threshold high-byte bucket 中真实匹配 leaf 数量。

正确性要求:

- 新旧 selected_prefix、count、candidates 逐元素完全相同。
- threshold_key、remaining、candidate_count 在 debug 模式下一致。
- 支持 capacity<=4096 fast path, CUDA Graph 可捕获。
- 不要顺手修改 kThreads、radix width、fallback 6-pass、final scan 或 expansion。

请先给出最小 patch, 再给出单元测试和 benchmark 代码; 如果任何 warp mask 使用存在潜在未定义行为, 优先保证 mask 中所有 named lanes 执行相同 sync intrinsic。

8. 实现风险清单

风险	解决办法
Warp mask 使用错误	__match_any_sync / __reduce_add_sync / __shfl_sync 的 mask 中，所有 named lanes 必须参与相同 intrinsic；不要在 named lanes 中让一部分提前 return。
continue 导致 ballot 分歧	Phase B 必须把 length<=0 变成 match=false，不能让部分 active lanes 在 ballot 之前 continue。
candidate_count 只计已写入项	错误。即使未来 pos>=kFastMaxLeaves，也应保持 candidate_count 代表真实匹配数；当前 fast path capacity<=4096 因而正常不会溢出。
错误依赖 threshold_candidates 顺序	Phase C 不应依赖物理顺序。稳定 tie 仍由 final logical scan 保证。
sm75 编译/运行问题	__reduce_add_sync 仅在 8.x+ 支持；必须有编译期架构 fallback，或明确项目最低架构为 sm80+ 后删除 fallback。
优化后更慢	如果 score 几乎均匀落在 256 bins，match_any/reduce 的额外指令可能抵消 atomic 减少；因此必须按真实 score 分布 benchmark。

9. 预期结果

这两项修改都不改变 weighted radix select 的算法复杂度，也不改变阈值定义。它们针对的是 NVIDIA warp 执行模型下的原子竞争：第一项把“同 warp、同 bin”的加权 histogram 更新合并；第二项把 threshold bucket 的位置预留从 per-leaf atomic 变成 per-warp atomic。

目标：同样的 threshold、同样的 candidates、更少的 atomic contention。

10. 依据与参考

- 当前实现：weighted_select(1).cu。fast path 使用 weighted FP32 high-byte histogram、threshold bucket packing、随后三个 radix refinement pass；最终 logical scan 负责 stable score-desc/index-asc 与 partial crossing leaf。
- NVIDIA CUDA Programming Guide：__match_any_sync 支持 compute capability 7.x+；warp reduce intrinsics（包括 __reduce_add_sync）支持 compute capability 8.x+；__ballot_sync 返回 predicate 为真的 lane mask。所有被 sync mask 命名且未退出的线程必须执行对应 intrinsic，否则行为未定义。